

LAPORAN PRAKTIKUM PERTEMUAN 5
PEMROGRAMAN MOBILE



Dosen Pengampu:

Jefry Sunupurwa Asri, S.Kom., M.Kom.

Disusun Oleh:

Ramsay Abelson 20240801042

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
TANGERANG
2026

PORAN PRAKTIKUM PERTEMUAN 5

Penyimpanan Lokal (SharedPreferences dan SQLite)

1. Tujuan Pembelajaran

1. Memilih jenis penyimpanan lokal yang tepat: key-value, database, atau berkas.
2. Menyimpan dan membaca pengaturan sederhana dengan `shared_preferences`.
3. Merancang tabel dan membuat database SQLite dengan `sqflite`.
4. Mengimplementasikan operasi CRUD (Create, Read, Update, Delete) dan menghubungkannya dengan UI.
5. Memastikan data tetap ada setelah aplikasi ditutup dan dibuka kembali.

2. Alat dan Bahan

1. Flutter SDK dan Visual Studio Code sebagai editor.
2. Emulator Android (`emulator-5554`) untuk menjalankan aplikasi.
3. Paket `shared_preferences`, `sqflite`, dan `path`.
4. Koneksi internet untuk mengunduh paket.

3. Teori Singkat

Data di memori (variabel dan State) hilang begitu aplikasi ditutup. Agar data bertahan, aplikasi menyimpannya di penyimpanan perangkat (persistence). Pilihan penyimpanan lokal dirangkum pada Tabel 1.

Tabel 1. Jenis penyimpanan lokal

Pilihan	Cocok untuk	Paket
Key-value	Pengaturan kecil: tema, nama, status login, angka terakhir	<code>shared_preferences</code>
Database SQLite	Data terstruktur dan banyak: catatan, transaksi, daftar produk	<code>sqflite</code> , <code>path</code>
Berkas	Dokumen, gambar, ekspor/impor data	<code>path_provider</code> , <code>dart:io</code>

Setiap operasi CRUD memiliki padanan perintah SQL dan method pada `sqflite` (Tabel 2). Query sebaiknya tidak disusun dengan menyambung teks dari pengguna; gunakan tanda tanya sebagai placeholder, misalnya `where: 'id = ?'`, `whereArgs: [id]`, agar aman dari SQL injection.

Tabel 2. Padanan CRUD, SQL, dan `sqflite`

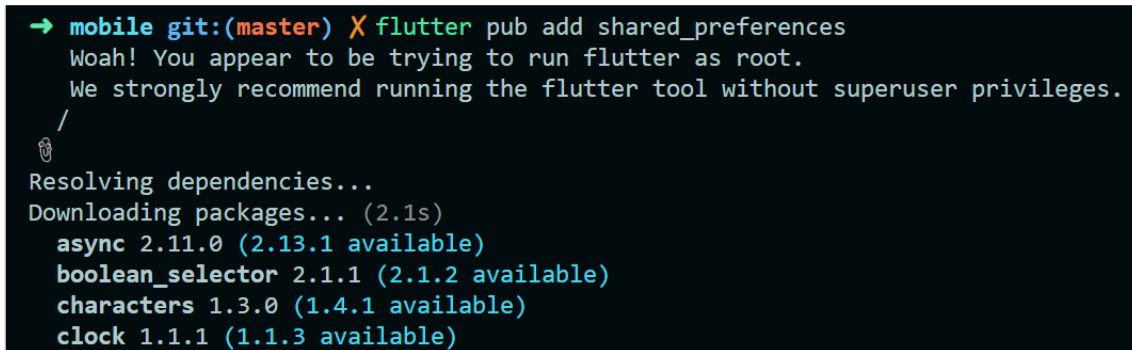
CRUD	SQL	Method <code>sqflite</code>
Create	INSERT	<code>db.insert()</code>
Read	SELECT	<code>db.query()</code>
Update	UPDATE	<code>db.update()</code>

CRUD	SQL	Method sqflite
Delete	DELETE	db.delete()

4. Langkah Praktikum

4.1 Bagian A: SharedPreferences (Pengaturan Sederhana)

Paket `shared_preferences` dipasang dengan perintah `flutter pub add shared_preferences` (Gambar 1). Aplikasi pada bagian ini menyimpan tiga data: nama (`String`, kunci nama), pilihan mode gelap (`bool`, kunci gelap), dan jumlah aplikasi dibuka (`int`, kunci bukaKe).



```

→ mobile git:(master) X flutter pub add shared_preferences
Woah! You appear to be trying to run flutter as root.
We strongly recommend running the flutter tool without superuser privileges.
/
Resolving dependencies...
Downloading packages... (2.1s)
  async 2.11.0 (2.13.1 available)
  boolean_selector 2.1.1 (2.1.2 available)
  characters 1.3.0 (1.4.1 available)
  clock 1.1.1 (1.1.3 available)

```

Gambar 1. Pemasangan paket `shared_preferences`

Kode lengkap `main.dart` untuk Bagian A sampai D ditampilkan per bagian. Pada tahap Bagian A, `home` pada `MyApp` berisi `PengaturanPage`; pada kode akhir di bawah, `home` sudah diganti menjadi `CatatanPage` sesuai Bagian C.

Kode 1. Import, fungsi main, dan MyApp

```

import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      home: const CatatanPage(),
    );
  }
}

```

Kode 2. PengaturanPage (SharedPreferences)

```

class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});
  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();

```

```

bool _gelap = false;
int _bukaKe = 0;
@override
void initState() {
  super.initState();
  _muat();
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

Future<void> _muat() async {
  final prefs = await SharedPreferences.getInstance();
  if (!mounted) return;
  setState(() {
    _controller.text = prefs.getString('nama') ?? '';
    _gelap = prefs.getBool('gelap') ?? false;
    _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
  });
  await prefs.setInt('bukaKe', _bukaKe);
}

Future<void> simpanNama() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('nama', _controller.text.trim());
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Nama disimpan')),
  );
}

Future<void> _ubahTema(bool nilai) async {
  setState(() => _gelap = nilai);
  final prefs = await SharedPreferences.getInstance();
  await prefs.setBool('gelap', nilai);
}

@override
Widget build(BuildContext context) {
  return Theme(
    data: ThemeData(
      brightness: _gelap ? Brightness.dark : Brightness.light,
      colorSchemeSeed: Colors.blue,
      useMaterial3: true,
    ),
    child: Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: _simpanNama,
              child: const Text('Simpan nama'),
            ),
            SwitchListTile(
              title: const Text('Mode gelap'),
              value: _gelap,
              onChanged: _ubahTema,
            ),
            const SizedBox(height: 12),
            Text('Aplikasi dibuka ke-$_bukaKe kali'),
          ],
        ),
      ),
    ),
  ),
)

```

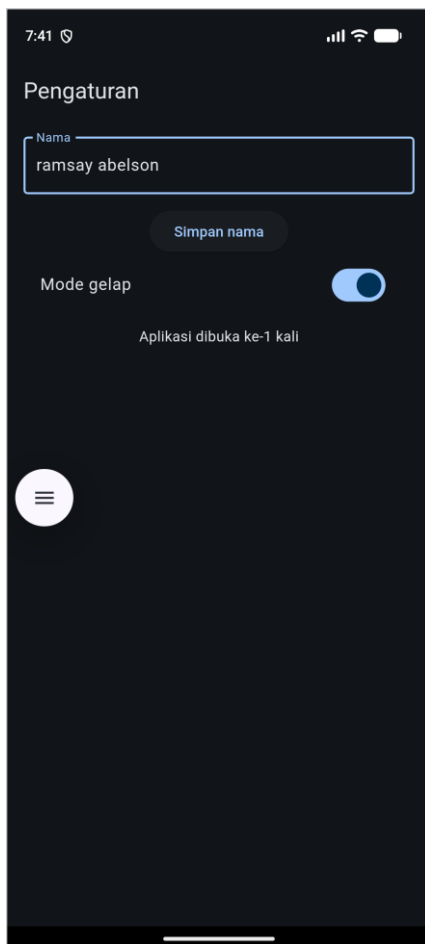
```

    );
  }
}

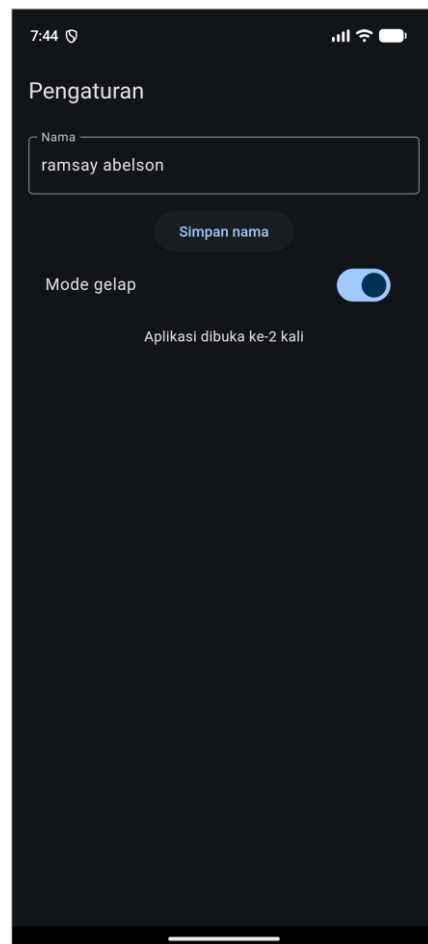
```

Method `_muat()` membaca data dengan nilai bawaan (`''`, `false`, `0`) lalu menambah hitungan buka dan menyimpannya kembali. Method `_simpanNama()` dan `_ubahTema()` menulis data setiap kali ada perubahan.

Checkpoint: nama diisi, mode gelap diaktifkan, kemudian aplikasi dihentikan sepenuhnya dan dijalankan lagi. Hasilnya, nama dan mode gelap tetap tersimpan, dan hitungan bertambah dari ke-1 menjadi ke-2 (Gambar 2 dan Gambar 3).



Gambar 2. Pembukaan pertama (dibuka ke-1 kali)



Gambar 3. Setelah dijalankan ulang (dibuka ke-2 kali)

4.2 Bagian B: Menyiapkan SQLite

Paket dipasang dengan `flutter pub add sqflite path` (Gambar 4). Model Catatan mewakili satu baris tabel, sedangkan `DbHelper` membuka database `catatan.db` dan menyediakan method `tambah`, `semua`, `ubah`, dan `hapus`. Tabel `catatan` dibuat pada `onCreate` dengan kolom `id` (primary key `AUTOINCREMENT`), `judul`, dan `isi`.

```

→ mobile git:(master) X flutter pub add sqflite path
Woah! You appear to be trying to run flutter as root.
We strongly recommend running the flutter tool without superuser privileges.
/
🐛
Resolving dependencies...
Downloading packages...
  async 2.11.0 (2.13.1 available)

```

Gambar 4. Pemasangan paket sqflite dan path

Kode 3. Model Catatan dan DbHelper

```

class Catatan {
  final int? id;
  final String judul;
  final String isi;

  const Catatan({this.id, required this.judul, required this.isi});

  Map<String, Object?> toMap() => {'id': id, 'judul': judul, 'isi': isi};

  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
    id: m['id'] as int,
    judul: m['judul'] as String,
    isi: m['isi'] as String,
  );
}

class DbHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'catatan.db');
    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE catatan('
            'id INTEGER PRIMARY KEY AUTOINCREMENT, '
            'judul TEXT NOT NULL, '
            'isi TEXT NOT NULL)',
        );
      },
    );
    return _db!;
  }

  static Future<int> tambah(Catatan c) async {
    final db = await database;
    return db.insert('catatan', c.toMap());
  }

  static Future<List<Catatan>> semua() async {
    final db = await database;
    final rows = await db.query('catatan', orderBy: 'id DESC');
    return rows.map(Catatan.fromMap).toList();
  }

  static Future<int> ubah(Catatan c) async {
    final db = await database;
    return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs: [c.id]);
  }

  static Future<int> hapus(int id) async {
    final db = await database;
    return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
  }
}

```

Nilai `id` bernilai `null` saat catatan baru dibuat, lalu diisi otomatis oleh SQLite. Seluruh query memakai placeholder ? dengan `whereArgs`.

4.3 Bagian C: Halaman Daftar Catatan (Read dan Delete)

`CatatanPage` menampilkan daftar catatan memakai `FutureBuilder`. Ada empat kondisi yang ditangani: memuat (`CircularProgressIndicator`), galat, data kosong ("Belum ada catatan"), dan daftar data. Ikon sampah memanggil `DbHelper.hapus()` lalu `_muat()`.

Kode 4. CatatanPage

```
class CatatanPage extends StatefulWidget {
  const CatatanPage({super.key});
  @override
  State<CatatanPage> createState() => CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
  late Future<List<Catatan>> _future;
  @override
  void initState() {
    super.initState();
    _future = DbHelper.semua();
  }

  void _muat() {
    setState(() {
      _future = DbHelper.semua();
    });
  }

  Future<void> _buka([Catatan? catatan]) async {
    await Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => FormCatatanPage(catatan: catatan)),
    );
    if (!mounted) return;
    _muat();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Saya')),
      body: FutureBuilder<List<Catatan>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(child: Text('Galat: ${snapshot.error}'));
          }
          final data = snapshot.data!;
          if (data.isEmpty) {
            return const Center(child: Text('Belum ada catatan'));
          }
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final c = data[i];
              return ListTile(
                title: Text(c.judul),
                subtitle:
                  Text(c.isi, maxLines: 2, overflow: TextOverflow.ellipsis),
                onTap: () => _buka(c),
                trailing: IconButton(
                  icon: const Icon(Icons.delete),
                  onPressed: () async {
                    await DbHelper.hapus(c.id!);
                    _muat();
                  },
                ),
              );
            },
          );
        },
      ),
    );
  }
}
```

```

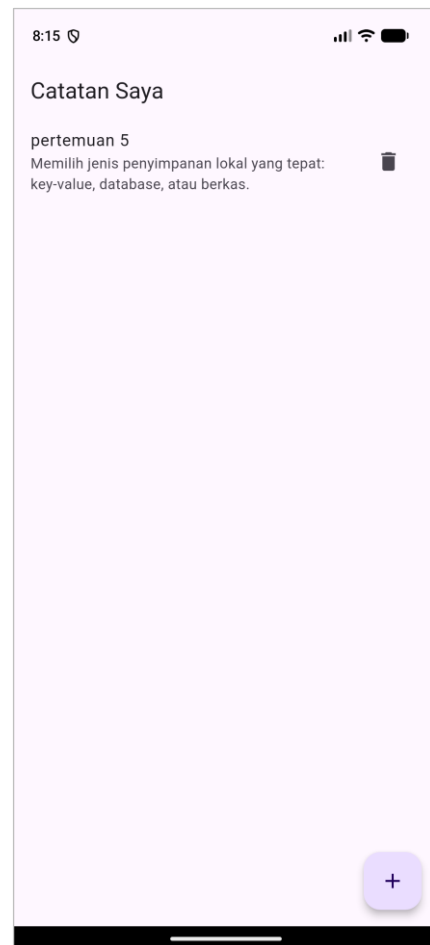
        ),
      ),
    },
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () => _buka(),
    child: const Icon(Icons.add),
  ),
),
);
}
}

```

Gambar 5 menunjukkan tampilan saat database masih kosong, dan Gambar 6 menunjukkan daftar setelah catatan "pertemuan 5" ditambahkan.



Gambar 5. Daftar kosong



Gambar 6. Daftar berisi catatan

4.4 Bagian D: Form Tambah dan Ubah Catatan (Create dan Update)

FormCatatanPage dipakai untuk dua keperluan: bila parameter catatan kosong berarti **tambah** (`DbHelper.tambah`), bila terisi berarti **ubah** (`DbHelper.ubah`). Form memiliki validasi sehingga judul dan isi wajib diisi. Setelah kembali dari form, `_buka()` pada `CatatanPage` memanggil `muat()` agar daftar diperbarui.

Kode 5. FormCatatanPage

```
class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;
  const FormCatatanPage({super.key, this.catatan});
  @override
  State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul;
  late final TextEditingController _isi;
  @override
  void initState() {
    super.initState();
    _judul = TextEditingController(text: widget.catatan?.judul ?? '');
    _isi = TextEditingController(text: widget.catatan?.isi ?? '');
  }

  @override
  void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final c = Catatan(
      id: widget.catatan?.id,
      judul: _judul.text.trim(),
      isi: _isi.text.trim(),
    );
    if (widget.catatan == null) {
      await DbHelper.tambah(c);
    } else {
      await DbHelper.ubah(c);
    }
    if (!mounted) return;
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    final baru = widget.catatan == null;
    return Scaffold(
      appBar: AppBar(title: Text(baru ? 'Catatan Baru' : 'Ubah Catatan')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judul,
              decoration: const InputDecoration(
                labelText: 'Judul',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _isi,
              maxLines: 6,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                (v == null || v.trim().isEmpty) ? 'Isi wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

```

    },
  },
};
}

```

Gambar 7 menunjukkan form dalam mode ubah: mengetuk catatan "pertemuan 5" membuka form yang sudah terisi judul dan isinya.

The screenshot shows a mobile application interface for editing a note. At the top, the status bar displays the time 8:16 and various icons. The app's title bar shows a back arrow and the text 'Ubah Catatan'. Below this, there are two main input fields: 'Judul' (Title) and 'Isi catatan' (Note content). The 'Judul' field contains the text 'pertemuan 5'. The 'Isi catatan' field contains a list of three items: 'Memilih jenis penyimpanan lokal yang tepat: key-value, database, atau berkas.', '2. Menyimpan dan membaca pengaturan sederhana dengan shared_preferences.', and '3. Merancang tabel dan membuat database SQLite dengan sqflite.'. At the bottom of the form is a 'Simpan' (Save) button.

Gambar 7. Form Ubah Catatan

5. Latihan Mandiri

Keempat latihan mandiri diimplementasikan pada satu berkas `main.dart` (Kode 6). Setiap latihan ditandai dengan komentar LATIHAN pada kode.

1. **Dialog konfirmasi hapus.** Ikon sampah menampilkan `AlertDialog` "Hapus catatan ini?" dengan tombol Batal dan Hapus. Data hanya dihapus bila pengguna memilih Hapus (Gambar 10).
2. **Kolom pencarian.** `TextField` "Cari judul" memanggil `_muat()` setiap teks berubah. Pada `DbHelper.semua()`, filter memakai `where: 'judul LIKE ?'` dan `whereArgs: ['%$kata%']` (Gambar 9).
3. **Kolom dibuat (tanggal dibuat).** Model `Catatan` mendapat properti `dibuat`, `version` dinaikkan dari 1 ke 2, dan `onUpgrade` menjalankan `ALTER TABLE catatan ADD`

COLUMN dibuat TEXT. Tanggal diisi dengan

`DateTime.now().toIso8601String()` saat catatan baru dibuat dan dipertahankan saat catatan diubah. Tanggal tampil sebagai "Dibuat: 2026-10-05 20:29" (Gambar 8).

Bila version tidak dinaikkan, `onUpgrade` tidak dijalankan sehingga kolom dibuat tidak pernah ditambahkan pada database lama dan muncul galat `no such column`.

4. **Pilihan urutan.** Ikon urutan pada AppBar (`PopupMenuButton`) menyediakan pilihan Terbaru dan Terlama. Pilihan disimpan dengan `shared_preferences` (kunci urutan), dibaca kembali di `_muatAwal()`, dan diterapkan pada `orderBy (id DESC` untuk terbaru, `id ASC` untuk terlama).

Kode 6. main.dart Latihan Mandiri

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      home: const CatatanPage(),
    );
  }
}

// ===== BAGIAN A: SharedPreferences =====

class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});

  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();
  bool _gelap = false;
  int _bukaKe = 0;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
    final prefs = await SharedPreferences.getInstance();
    if (!mounted) return;
    setState(() {
      _controller.text = prefs.getString('nama') ?? '';
      _gelap = prefs.getBool('gelap') ?? false;
      _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
    });
    await prefs.setInt('bukaKe', _bukaKe);
  }

  Future<void> _simpanNama() async {
```

```

        final prefs = await SharedPreferences.getInstance();
        await prefs.setString('nama', _controller.text.trim());
        if (!mounted) return;
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Nama disimpan')),
        );
    }

    Future<void> _ubahTema(bool nilai) async {
        setState(() => _gelap = nilai);
        final prefs = await SharedPreferences.getInstance();
        await prefs.setBool('gelap', nilai);
    }

    @override
    Widget build(BuildContext context) {
        return Theme(
            data: ThemeData(
                brightness: _gelap ? Brightness.dark : Brightness.light,
                colorSchemeSeed: Colors.blue,
                useMaterial3: true,
            ),
            child: Scaffold(
                appBar: AppBar(title: const Text('Pengaturan')),
                body: Padding(
                    padding: const EdgeInsets.all(16),
                    child: Column(
                        children: [
                            TextField(
                                controller: _controller,
                                decoration: const InputDecoration(
                                    labelText: 'Nama',
                                    border: OutlineInputBorder(),
                                ),
                            ),
                            const SizedBox(height: 12),
                            ElevatedButton(
                                onPressed: simpanNama,
                                child: const Text('Simpan nama'),
                            ),
                            SwitchListTile(
                                title: const Text('Mode gelap'),
                                value: _gelap,
                                onChanged: _ubahTema,
                            ),
                            const SizedBox(height: 12),
                            Text('Aplikasi dibuka ke-$_bukaKe kali'),
                        ],
                    ),
                ),
            ),
        );
    }
}

// ===== BAGIAN B: SQLite =====

class Catatan {
    final int? id;
    final String judul;
    final String isi;
    final String? dibuat; // LATIHAN 3: kolom tanggal dibuat

    const Catatan({this.id, required this.judul, required this.isi, this.dibuat});

    Map<String, Object?> toMap() =>
        {'id': id, 'judul': judul, 'isi': isi, 'dibuat': dibuat};

    factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
        id: m['id'] as int?,
        judul: m['judul'] as String,
        isi: m['isi'] as String,
        dibuat: m['dibuat'] as String?,
    );

    // Tampilan tanggal: 2026-10-05 14:30
    String get dibuatTampil {

```

```

        if (dibuat == null || dibuat!.length < 16) return '';
        return dibuat!.substring(0, 16).replaceFirst('T', ' ');
    }
}

class DbHelper {
    static Database? _db;

    static Future<Database> get database async {
        if (_db != null) return _db!;
        final path = p.join(await getDatabasesPath(), 'catatan.db');
        _db = await openDatabase(
            path,
            version: 2, // LATIHAN 3: dinaikkan dari 1 ke 2
            onCreate: (db, version) {
                return db.execute(
                    'CREATE TABLE catatan('
                    'id INTEGER PRIMARY KEY AUTOINCREMENT, '
                    'judul TEXT NOT NULL, '
                    'isi TEXT NOT NULL, '
                    'dibuat TEXT)',
                );
            },
            // LATIHAN 3: dijalankan bila database lama (versi 1) dibuka dengan version 2
            onUpgrade: (db, oldVersion, newVersion) async {
                if (oldVersion < 2) {
                    await db.execute('ALTER TABLE catatan ADD COLUMN dibuat TEXT');
                }
            },
        );
        return _db!;
    }

    static Future<int> tambah(Catatan c) async {
        final db = await database;
        return db.insert('catatan', c.toMap());
    }

    // LATIHAN 2 dan 4: pencarian (LIKE) dan urutan (orderBy)
    static Future<List<Catatan>> semua({
        String kata = '',
        String urutan = 'terbaru',
    }) async {
        final db = await database;
        final rows = await db.query(
            'catatan',
            where: kata.isEmpty ? null : 'judul LIKE ?',
            whereArgs: kata.isEmpty ? null : ['%$kata%'],
            orderBy: urutan == 'terlama' ? 'id ASC' : 'id DESC',
        );
        return rows.map(Catatan.fromMap).toList();
    }

    static Future<int> ubah(Catatan c) async {
        final db = await database;
        return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs: [c.id]);
    }

    static Future<int> hapus(int id) async {
        final db = await database;
        return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
    }
}

// ===== BAGIAN C: Daftar Catatan =====

class CatatanPage extends StatefulWidget {
    const CatatanPage({super.key});

    @override
    State<CatatanPage> createState() => _CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
    late Future<List<Catatan>> _future;
    String _kata = '';
    String _urutan = 'terbaru';
}

```

```

@override
void initState() {
  super.initState();
  _future = _muatAwal();
}

// LATIHAN 4: baca pilihan urutan yang tersimpan, lalu muat data
Future<List<Catatan>> _muatAwal() async {
  final prefs = await SharedPreferences.getInstance();
  final tersimpan = prefs.getString('urutan') ?? 'terbaru';
  if (mounted) setState(() => _urutan = tersimpan);
  return DbHelper.semua(kata: _kata, urutan: tersimpan);
}

void _muat() {
  setState(() {
    _future = DbHelper.semua(kata: _kata, urutan: _urutan);
  });
}

// LATIHAN 4: simpan pilihan urutan
Future<void> _ubahUrutan(String nilai) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('urutan', nilai);
  if (!mounted) return;
  _urutan = nilai;
  _muat();
}

Future<void> _buka([Catatan? catatan]) async {
  await Navigator.push(
    context,
    MaterialPageRoute(builder: (_) => FormCatatanPage(catatan: catatan)),
  );
  if (!mounted) return;
  _muat();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Catatan Saya'),
      actions: [
        PopupMenuButton<String>(
          icon: const Icon(Icons.sort),
          tooltip: 'Urutkan',
          initialValue: _urutan,
          onSelected: _ubahUrutan,
          itemBuilder: (_) => const [
            PopupMenuItem(value: 'terbaru', child: Text('Terbaru')),
            PopupMenuItem(value: 'terlama', child: Text('Terlama')),
          ],
        ),
      ],
    ),
    body: Column(
      children: [
        // LATIHAN 2: kolom pencarian
        Padding(
          padding: const EdgeInsets.fromLTRB(16, 12, 16, 4),
          child: TextField(
            decoration: const InputDecoration(
              labelText: 'Cari judul',
              prefixIcon: Icon(Icons.search),
              border: OutlineInputBorder(),
            ),
            onChanged: (v) {
              _kata = v.trim();
              _muat();
            },
          ),
        ),
        Expanded(
          child: FutureBuilder<List<Catatan>>(
            future: _future,

```



```
// ===== BAGIAN D: Form Tambah/Ubah =====

class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;
  const FormCatatanPage({super.key, this.catatan});

  @override
  State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul;
  late final TextEditingController _isi;

  @override
  void initState() {
    super.initState();
    _judul = TextEditingController(text: widget.catatan?.judul ?? '');
    _isi = TextEditingController(text: widget.catatan?.isi ?? '');
  }

  @override
  void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

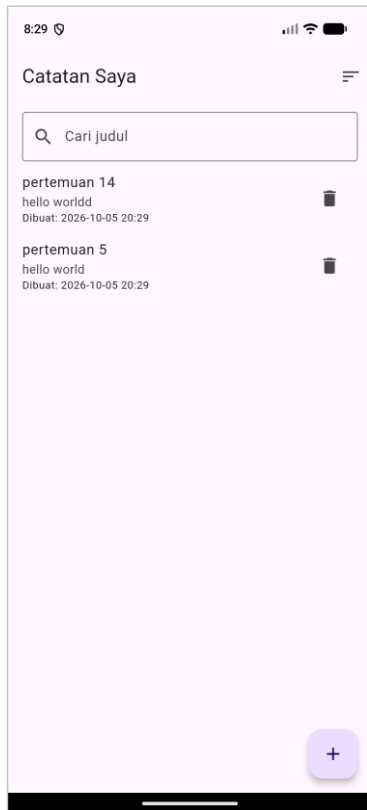
  Future<void> _simpan() async {
    if (!formKey.currentState!.validate()) return;
    final c = Catatan(
      id: widget.catatan?.id,
      judul: _judul.text.trim(),
      isi: _isi.text.trim(),
      // LATIHAN 3: isi tanggal saat catatan baru dibuat, pertahankan saat ubah
      dibuat: widget.catatan?.dibuat ?? DateTime.now().toIso8601String(),
    );
    if (widget.catatan == null) {
      await DbHelper.tambah(c);
    } else {
      await DbHelper.ubah(c);
    }
    if (!mounted) return;
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    final baru = widget.catatan == null;
    return Scaffold(
      appBar: AppBar(title: Text(baru ? 'Catatan Baru' : 'Ubah Catatan')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judul,
              decoration: const InputDecoration(
                labelText: 'Judul',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _isi,
              maxLines: 6,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                (v == null || v.trim().isEmpty) ? 'Isi wajib diisi' : null,
            ),
          ],
        ),
      ),
    );
  }
}
```

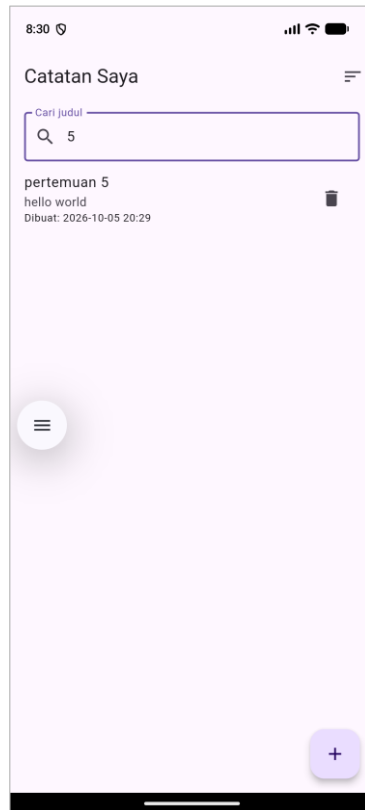
```

    ),
    const SizedBox(height: 16),
    ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
  ],
),
),
);
}
}

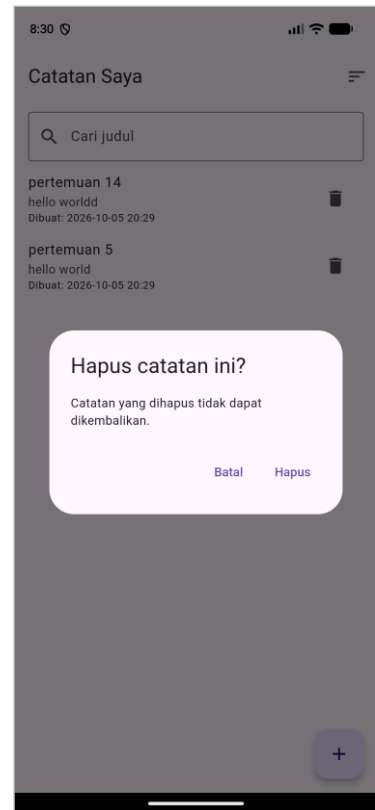
```



Gambar 8. Daftar dengan kolom pencarian, ikon urutan, dan tanggal dibuat



Gambar 9. Pencarian dengan kata "5"



Gambar 10. Dialog konfirmasi hapus

6. Tugas: Aplikasi Pencatat Pengeluaran

Aplikasi Pencatat Pengeluaran dibuat dengan database pengeluaran.db dan tabel pengeluaran berisi kolom id, nama, jumlah (integer), kategori, dan tanggal. Pemenuhan ketentuan tugas dirangkum pada Tabel 3.

Tabel 3. Pemenuhan ketentuan tugas

Ketentuan	Implementasi
Tabel pengeluaran	DatabasePengeluaran membuat tabel pengeluaran pada onCreate.
CRUD lengkap	Daftar (semua), tambah dan ubah (simpan), hapus (hapus) dengan dialog konfirmasi. Menu ubah/hapus tersedia pada tiap item.

Ketentuan	Implementasi
Validasi form	Nama wajib diisi, jumlah harus angka bulat lebih dari 0, kategori dipilih dari dropdown (kategoriPengeluaran).
Total pengeluaran	Dihitung dengan <code>SELECT SUM(jumlah) AS total FROM pengeluaran</code> dan ditampilkan pada kartu di halaman utama.
Pengaturan SharedPreferences	Pilihan mode gelap disimpan dengan kunci modeGelap.
Data bertahan	Data tersimpan di SQLite dan tetap ada setelah aplikasi dijalankan ulang (Gambar 13).

Kode 7. main.dart Tugas Pencatat Pengeluaran

```
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:shared_preferences/shared_preferences.dart';
import 'package:sqflite/sqflite.dart';

void main() => runApp(const AplikasiPengeluaran());

class AplikasiPengeluaran extends StatefulWidget {
  const AplikasiPengeluaran({super.key});

  @override
  State<AplikasiPengeluaran> createState() => _AplikasiPengeluaranState();
}

class _AplikasiPengeluaranState extends State<AplikasiPengeluaran> {
  bool _modeGelap = false;

  @override
  void initState() {
    super.initState();
    SharedPreferences.getInstance().then((prefs) {
      if (mounted) {
        setState(() => _modeGelap = prefs.getBool('modeGelap') ?? false);
      }
    });
  }

  Future<void> _ubahTema(bool gelap) async {
    setState(() => _modeGelap = gelap);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('modeGelap', gelap);
  }

  @override
  Widget build(BuildContext context) => MaterialApp(
    title: 'Pencatat Pengeluaran',
    theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
    darkTheme: ThemeData(
      colorSchemeSeed: Colors.teal,
      brightness: Brightness.dark,
      useMaterial3: true),
    themeMode: _modeGelap ? ThemeMode.dark : ThemeMode.light,
    home:
      HalamanPengeluaran(modeGelap: _modeGelap, onTemaChanged: _ubahTema),
  );
}

const kategoriPengeluaran = [
  'Makanan',
  'Transportasi',
  'Belanja',
  'Tagihan',
  'Kesehatan',
  'Hiburan',
  'Lainnya'
];
```

```

class Pengeluaran {
    const Pengeluaran(
        {this.id,
        required this.nama,
        required this.jumlah,
        required this.kategori,
        required this.tanggal});
    final int? id;
    final String nama;
    final int jumlah;
    final String kategori;
    final String tanggal;

    Map<String, Object?> toMap() => {
        'nama': nama,
        'jumlah': jumlah,
        'kategori': kategori,
        'tanggal': tanggal
    };
    factory Pengeluaran.fromMap(Map<String, Object?> map) => Pengeluaran(
        id: map['id'] as int,
        nama: map['nama'] as String,
        jumlah: map['jumlah'] as int,
        kategori: map['kategori'] as String,
        tanggal: map['tanggal'] as String,
    );
}

class DatabasePengeluaran {
    static Database? _database;
    static Future<Database> get database async {
        if (_database != null) return _database!;
        final path = p.join(await getDatabasesPath(), 'pengeluaran.db');
        _database =
            await openDatabase(path, version: 1, onCreate: (db, version) async {
                await db.execute('''CREATE TABLE pengeluaran(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    nama TEXT NOT NULL,
                    jumlah INTEGER NOT NULL,
                    kategori TEXT NOT NULL,
                    tanggal TEXT NOT NULL
                )''');
            });
        return _database!;
    }

    static Future<List<Pengeluaran>> semua() async {
        final rows = await (await database)
            .query('pengeluaran', orderBy: 'tanggal DESC, id DESC');
        return rows.map(Pengeluaran.fromMap).toList();
    }

    static Future<int> total() async {
        final rows = await (await database)
            .rawQuery('SELECT SUM(jumlah) AS total FROM pengeluaran');
        return (rows.first['total'] as int?) ?? 0;
    }

    static Future<void> simpan(Pengeluaran item) async {
        final db = await database;
        if (item.id == null) {
            await db.insert('pengeluaran', item.toMap());
        } else {
            await db.update('pengeluaran', item.toMap(),
                where: 'id = ?', whereArgs: [item.id]);
        }
    }

    static Future<void> hapus(int id) async =>
        (await database).delete('pengeluaran', where: 'id = ?', whereArgs: [id]);
}

class HalamanPengeluaran extends StatefulWidget {
    const HalamanPengeluaran(
        {super.key, required this.modeGelap, required this.onTemaChanged});
    final bool modeGelap;

```

```

final ValueChanged<bool> onTemaChanged;

@override
State<HalamanPengeluaran> createState() => _HalamanPengeluaranState();
}

class _HalamanPengeluaranState extends State<HalamanPengeluaran> {
  late Future<List<Pengeluaran>> _data;
  late Future<int> _total;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  void _muat() {
    _data = DatabasePengeluaran.semua();
    _total = DatabasePengeluaran.total();
  }

  Future<void> _tambahUbah([Pengeluaran? item]) async {
    final tersimpan = await Navigator.push<bool>(context,
      MaterialPageRoute(builder: (_) => FormPengeluaran(item: item)));
    if (tersimpan == true && mounted) setState(_muat);
  }

  String _rupiah(int nilai) =>
    'Rp ${nilai.toString().replaceAllMapped(RegExp(r'\B(?=(\d{3})+(?!\d))'), (_) =>
      '.)')}';

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(
      title: const Text('Pengeluaran Saya'),
      actions: [
        const Padding(
          padding: EdgeInsets.only(left: 8),
          child: Center(child: Text('Gelap'))),
        Switch(value: widget.modeGelap, onChanged: widget.onTemaChanged),
      ],
    ),
    body: Column(children: [
      FutureBuilder<int> (
        future: _total,
        builder: (context, snapshot) => Card(
          margin: const EdgeInsets.all(16),
          child: ListTile(
            leading:
              const Icon(Icons.account_balance_wallet_outlined),
            title: const Text('Total pengeluaran'),
            subtitle: Text(snapshot.hasData
              ? _rupiah(snapshot.data!)
              : 'Menghitung...'),
          ),
        ),
      ),
      Expanded(
        child: FutureBuilder<List<Pengeluaran>>(
          future: _data,
          builder: (context, snapshot) {
            if (snapshot.hasError) {
              return Center(child: Text('Gagal memuat: ${snapshot.error}'));
            }
            if (!snapshot.hasData) {
              return const Center(child: CircularProgressIndicator());
            }
            final items = snapshot.data!;
            if (items.isEmpty) {
              return const Center(
                child:
                  Text('Belum ada pengeluaran. Tekan + untuk menambah.'));
            }
            return ListView.builder(
              itemCount: items.length,
              itemBuilder: (context, index) {
                final item = items[index];
                return ListTile(

```



```

        _nama.dispose();
        _jumlah.dispose();
        super.dispose();
    }

    String get _tanggalText =>
        '${_tanggal.year.toString().padLeft(4, '0')}-${_tanggal.month.toString().padLeft(2, '0')}-${_tanggal.day.toString().padLeft(2, '0')}';

    Future<void> _pilihTanggal() async {
        final pilihan = await showDatePicker(
            context: context,
            initialDate: _tanggal,
            firstDate: DateTime(2000),
            lastDate: DateTime(2100));
        if (pilihan != null) setState(() => _tanggal = pilihan);
    }

    Future<void> _simpan() async {
        if (!_formKey.currentState!.validate()) return;
        await DatabasePengeluaran.simpan(Pengeluaran(
            id: widget.item?.id,
            nama: _nama.text.trim(),
            jumlah: int.parse(_jumlah.text.trim()),
            kategori: _kategori,
            tanggal: tanggalText,
        ));
        if (mounted) Navigator.pop(context, true);
    }

    @override
    Widget build(BuildContext context) => Scaffold(
        appBar: AppBar(
            title: Text(widget.item == null
                ? 'Tambah Pengeluaran'
                : 'Ubah Pengeluaran')),
        body: Form(
            key: formKey,
            child: ListView(padding: const EdgeInsets.all(16), children: [
                TextFormField(
                    controller: _nama,
                    decoration: const InputDecoration(
                        labelText: 'Nama pengeluaran',
                        border: OutlineInputBorder()),
                    validator: (value) => value == null || value.trim().isEmpty
                        ? 'Nama wajib diisi'
                        : null,
                ),
                const SizedBox(height: 12),
                TextFormField(
                    controller: _jumlah,
                    keyboardType: TextInputType.number,
                    decoration: const InputDecoration(
                        labelText: 'Jumlah (rupiah)',
                        prefixText: 'Rp ',
                        border: OutlineInputBorder()),
                    validator: (value) {
                        final angka = int.tryParse(value?.trim() ?? '');
                        if (angka == null) return 'Jumlah harus berupa angka bulat';
                        if (angka <= 0) return 'Jumlah harus lebih dari 0';
                        return null;
                    },
                ),
                const SizedBox(height: 12),
                DropdownButtonFormField<String>(
                    value: _kategori,
                    decoration: const InputDecoration(
                        labelText: 'Kategori', border: OutlineInputBorder()),
                    items: kategoriPengeluaran
                        .map((kategori) => DropdownMenuItem(
                            value: kategori, child: Text(kategori)))
                        .toList(),
                    onChanged: (value) {
                        if (value != null) setState(() => _kategori = value);
                    },
                ),
                const SizedBox(height: 12),
            ]),
    );

```

```

        OutlinedButton.icon(
          onPressed: _pilihTanggal,
          icon: const Icon(Icons.calendar_today),
          label: Text('Tanggal: $_tanggalText'),
          const SizedBox(height: 20),
          FilledButton.icon(
            onPressed: _simpan,
            icon: const Icon(Icons.save),
            label: const Text('Simpan')),
        )),
      );
    }
  }
}

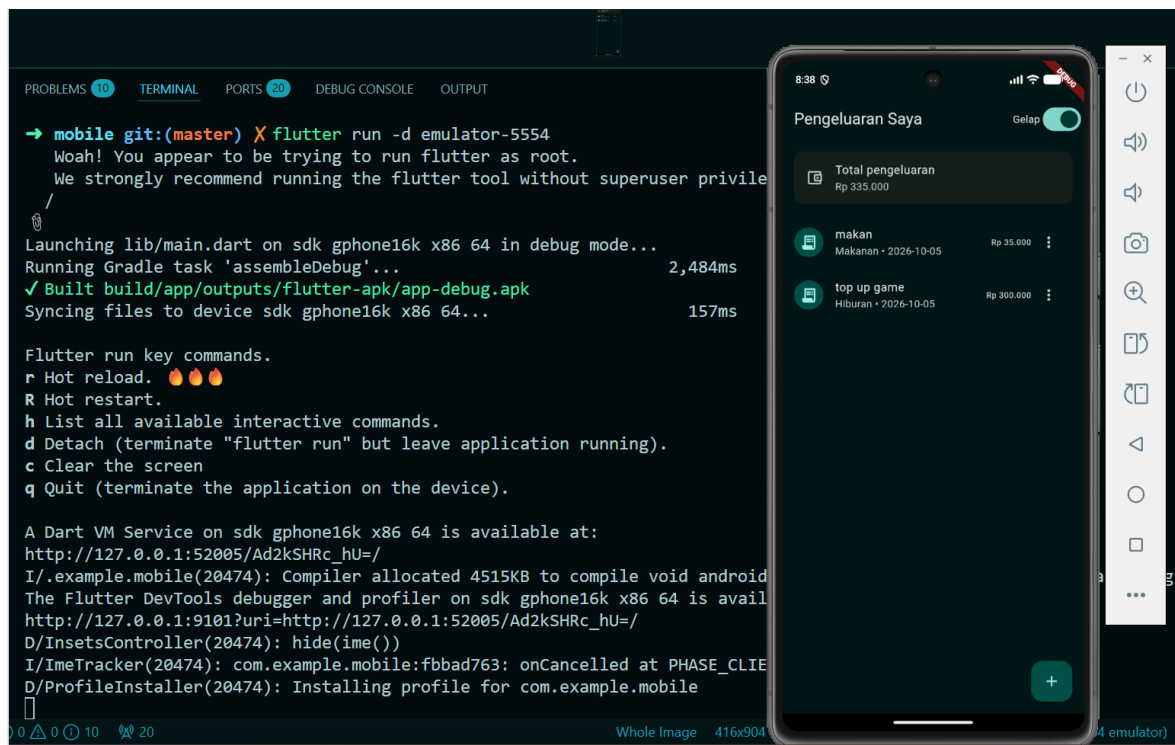
```

Gambar 11 menunjukkan form tambah pengeluaran dengan nama "top up game", jumlah 300000, kategori Hiburan, dan tanggal 2026-10-05. Gambar 12 menunjukkan halaman utama dalam mode gelap: dua pengeluaran ("makan" Rp 35.000 dan "top up game" Rp 300.000) dengan total pengeluaran Rp 335.000.

Gambar 11. Form tambah pengeluaran

Gambar 12. Daftar pengeluaran dan total

Bukti data bertahan: aplikasi dijalankan ulang dengan perintah `flutter run -d emulator-5554`. Setelah aplikasi terbuka kembali, kedua pengeluaran, total Rp 335.000, dan mode gelap (Gelap aktif) masih tampil (Gambar 13).



Gambar 13. Bukti data bertahan setelah aplikasi dijalankan ulang

7. Jawaban Pertanyaan Refleksi

1. **Mengapa `shared\_preferences` kurang cocok untuk ratusan catatan?**
`shared_preferences` dirancang untuk pasangan key-value sederhana dan berukuran kecil. Ia tidak mendukung query, pencarian, pengurutan, maupun struktur tabel, sehingga data yang banyak akan sulit dikelola dan tidak efisien. Pilihan yang lebih baik adalah database SQLite dengan `sqflite`.
2. **Mengapa query memakai placeholder `?` dan `whereArgs`?** Dengan placeholder, input pengguna diperlakukan sebagai data, bukan bagian dari perintah SQL. Hal ini mencegah SQL injection, sedangkan menyambung teks langsung memungkinkan pengguna menyisipkan perintah SQL berbahaya.
3. **Mengapa daftar perlu dimuat ulang setelah kembali dari form?** `FutureBuilder` memegang hasil query sebelumnya (`_future`) dan tidak otomatis mengetahui bahwa isi database berubah. Pada kode ini, `_buka()` menunggu `Navigator.push` selesai (`await`), lalu memanggil `_muat()` yang mengisi `_future` dengan `DbHelper.semua()` di dalam `setState`.
4. **Fungsi `onCreate` dan `onUpgrade`?** `onCreate` dijalankan sekali ketika database belum ada dan baru dibuat, dipakai untuk membuat tabel. `onUpgrade` dijalankan ketika database lama dibuka dengan nilai `version` yang lebih tinggi, dipakai untuk mengubah skema (misalnya `ALTER TABLE`) tanpa menghapus data.

8. Kesimpulan

Pada praktikum ini penyimpanan lokal Flutter berhasil diterapkan. `shared_preferences` cocok untuk pengaturan kecil seperti nama, mode gelap, hitungan buka, dan pilihan urutan, sedangkan SQLite melalui `sqflite` cocok untuk data terstruktur yang banyak. Operasi CRUD berhasil dihubungkan dengan UI, mencakup validasi form, pencarian, pengurutan, konfirmasi hapus, dan migrasi skema dengan `onUpgrade`. Aplikasi Pencatat Pengeluaran membuktikan bahwa data tetap tersimpan setelah aplikasi dijalankan ulang.